


INSTITUCIÓN UNIVERSITARIA

PASCUAL BRAVO®

INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO®

**Fecha**: 25/10/2026

**Estudiante**: Daniel Steven Pineda Zafra

**Docente:** Oralia Cortes Grajales

**Materia:** Bases de Datos I

# Tarea4. Comparar la implementación de un diccionario de datos físico en diferentes SGBD

# Comparación de las diferentes SGBD

## Introducción

La Red Social Pascualina busca mejorar la comunicación entre estudiantes mediante perfiles, publicaciones, intereses, habilidades, deportes, grupos, eventos y mentorías. Esta tarea continúa el trabajo anterior: no se diseñó una base de datos nueva, sino que se tomó la implementación de PostgreSQL como punto de partida para adaptarla a MySQL y SQLite.

El objetivo fue implementar el mismo modelo en tres SGBD, identificar sus diferencias y documentarlas en un diccionario genérico y tres diccionarios físicos. Además, se incorporó el atributo ubicacion_gps en evento para almacenar información semiestructurada y plantear un posible uso relacionado con IoT.

Cada implementación contiene 14 tablas, 56 atributos y 17 claves foráneas. Se conserva el propósito de las tablas, aunque algunos tipos de datos, identificadores y mecanismos de validación cambian entre motores. El proyecto no incluye una aplicación web ni captura real de señales GPS.

## Elección de los tres SGBD

Se trabajó con PostgreSQL, MySQL y SQLite para comparar tres implementaciones de una misma estructura. La elección mantuvo la continuidad del proyecto y permitió observar diferencias concretas al crear tablas, definir claves y guardar JSON. No se realizó una prueba de velocidad entre los motores.

| SGBD y herramienta | Razón de elección y ventaja observada | Dificultad o ajuste en este proyecto |
|---|---|---|
| PostgreSQL 18<br>pgAdmin 4 | Ya se había utilizado en la tarea anterior. Se conservaron UUID, IDENTITY y las restricciones CHECK. JSONB permitió ampliar evento sin sustituir el modelo. | La copia de la Tarea 4 debía mantenerse separada del entregable anterior. Fue necesario distinguir el cambio aplicado con ALTER TABLE del script de creación actualizado. |
| MySQL 9.7.4<br>MySQL Workbench | Permitió construir el mismo modelo y trabajar con un tipo JSON. AUTO_INCREMENT facilitó la generación de los identificadores numéricos. | Hubo que adaptar UUID, fechas, contenido y booleanos. El error 3823 obligó a sustituir dos CHECK por cuatro triggers. |
| SQLite<br>DB Browser for SQLite | El trabajo se organizó en un archivo .db. Se conservaron las relaciones y se utilizó TEXT con validación JSON, lo que facilitó comparar otra forma de almacenamiento. | Fue necesario activar las FK, controlar longitudes con CHECK y representar fechas y booleanos con los tipos elegidos para este motor. |

Para este proyecto, PostgreSQL fue el punto de partida, MySQL permitió analizar cambios de sintaxis y SQLite hizo más visible la diferencia entre declarar un tipo y establecer sus restricciones. Las ventajas anteriores se refieren al ejercicio realizado, no a una superioridad absoluta de un SGBD.

# Proceso de implementación

## PostgreSQL

**1. Recuperar la estructura.** Se conservó el script de las 14 tablas construido en la Tarea 3 y se creó una copia para la Tarea 4. Se mantuvieron las claves y las acciones referenciales del modelo.

**2. Añadir el dato semiestructurado.** En la base existente se ejecutó ALTER TABLE evento ADD COLUMN ubicacion_gps JSONB. En la copia del script se agregó el mismo atributo dentro de CREATE TABLE evento, para permitir una construcción desde cero.

**3. Verificar.** Se consultó information_schema.columns y se comprobó que ubicacion_gps figuraba como jsonb. Después se registraron un estudiante y un evento de prueba con información JSON.

## MySQL

**1. Preparar la base.** Se configuró el servidor local y la conexión en Workbench. Se creó red_social_pascualina con utf8mb4 y se seleccionó mediante USE red_social_pascualina.

**2. Adaptar las tablas**. Se cambió IDENTITY por AUTO_INCREMENT. Los UUID se representaron con CHAR(36) y UUID(); el contenido de las publicaciones, con MEDIUMTEXT y un CHECK de longitud. Se conservaron las referencias entre las tablas.

**3. Resolver restricciones y probar**. Se crearon dos triggers en conexion y dos en mentoria, para INSERT y UPDATE. Se verificó su existencia con SHOW TRIGGERS y se probaron inserciones con cédulas iguales. En evento se insertó JSON mediante JSON_OBJECT() y se consultaron sus coordenadas.

### SQLite

**1. Preparar el archivo.** Se creó red_social_pascualina.db en una carpeta con permisos de escritura. Se ejecutó PRAGMA foreign_keys = ON y se comprobó el valor 1. La activación debe verificarse en cada conexión, fuera de una transacción activa.

**2. Adaptar las tablas.** Se utilizó TEXT con límites mediante CHECK, INTEGER para booleanos restringidos a 0 y 1, y claves INTEGER PRIMARY KEY AUTOINCREMENT. Las columnas de las PK compuestas se declararon NOT NULL.

**3. Incorporar y probar JSON**. ubicacion_gps se declaró como TEXT con CHECK basado en json_valid(). Se registraron un estudiante y un evento de prueba y se consultaron la latitud y longitud con json_extract(). Se confirmaron 14 tablas del proyecto; sqlite_sequence es una tabla interna adicional.

## Cambios entre las implementaciones

La comparación muestra lo que está definido en los scripts, no todas las posibilidades que ofrece cada SGBD.

| Elemento | PostgreSQL | MySQL | SQLite |
|---|---|---|---|
| ID de catálogos y mentoría | INTEGER con GENERATED ALWAYS AS IDENTITY | INTEGER con AUTO_INCREMENT | INTEGER PRIMARY KEY AUTOINCREMENT |
| ID de publicación, grupo y evento | UUID y gen_random_uuid() | CHAR(36) y UUID() | INTEGER PRIMARY KEY AUTOINCREMENT |
| Contenido de publicación | VARCHAR(20000) | MEDIUMTEXT + CHECK CHAR_LENGTH < = 20000 | TEXT + CHECK LENGTH < = 20000 |
| Privacidad y respuesta | BOOLEAN | TINYINT(1) + CHECK IN (0, 1) | INTEGER + CHECK IN (0, 1) |
| Fecha y hora de evento | TIMESTAMP | DATETIME | TEXT; formato acordado YYYY-MM-DD HH:MM:SS |
| Ubicación GPS | JSONB | JSON | TEXT + CHECK json_valid() |
| Autoseguimiento y automentoría | CHECK | Triggers de INSERT y UPDATE con SIGNAL | CHECK |

El cambio de UUID a entero en SQLite fue una decisión de simplificación. No significa que SQLite sea incapaz de almacenar UUID como texto. Las FK de ingreso_grupo y asistencia se adaptaron a INTEGER.

La PK garantiza la unicidad. AUTOINCREMENT añade la no reutilización de identificadores automáticos de filas eliminadas y confirmadas; no garantiza una numeración sin saltos. Por eso se mantuvo como elección explícita, no como requisito para que exista una clave única.

En los tres scripts hay 17 FK con ON UPDATE CASCADE. Para borrados se definieron 15 CASCADE y 2 RESTRICT: no se elimina a un creador mientras mantenga grupos o eventos a su cargo. En las seis PK compuestas es única la combinación, no cada columna por separado.

## Diccionarios de datos

El diccionario genérico describe el modelo sin imponer una sintaxis particular. Los tres diccionarios físicos documentan la implementación real de cada motor. Su detalle se presenta como anexo independiente para no repetir las 14 tablas completas dentro del informe.

**1. Partir del modelo lógico.** Conservar las tablas, el significado de los atributos, las relaciones y la obligatoriedad. Incorporar ubicacion_gps como información opcional. En el genérico, describir los identificadores sin imponer UUID o una secuencia propia de un motor.

**2. Registrar cada implementación.** Para cada atributo, documentar nombre, tipo real, longitud o límite, PK, FK y referencia, unicidad, obligatoriedad, valor predeterminado, descripción y justificación.

**3. Completar las reglas**. En PostgreSQL, registrar UUID, IDENTITY, CHECK y JSONB. En MySQL, añadir AUTO_INCREMENT, CHAR(36), JSON y las reglas ejecutadas por triggers. En SQLite, registrar AUTOINCREMENT, los CHECK de longitud y de JSON, y los NOT NULL explícitos.

**4. Contrastar con el código ejecutado**. El diccionario debe describir la definición SQL, no solamente sus comentarios. Una columna integrante de una PK compuesta no se marca como individualmente única si el código no lo exige.

## JSON y posible aplicación en IoT

Se agregó ubicacion_gps a evento para complementar el lugar escrito del encuentro. El caso propuesto consiste en que una futura aplicación reciba una lectura de ubicación de un dispositivo conectado y guarde sus coordenadas y metadatos. En esta tarea se usaron datos de ejemplo; no se conectó un sensor real.


```
{
  "latitud": 6.2442,
  "longitud": -75.5812,
  "precision_metros": 10,
  "fuente": "GPS"
}
```


Las coordenadas son ilustrativas y no representan una ubicación verificada del campus. Latitud, longitud y precisión se guardan como números; fuente identifica el origen declarado de la lectura.

| Motor | Almacenamiento y consulta utilizados |
|---|---|
| PostgreSQL | ubicacion_gps JSONB. Se insertó un documento mediante conversión ::jsonb. El tipo valida la sintaxis y almacena una representación binaria del JSON. |
| MySQL | ubicacion_gps JSON. Se utilizó JSON_OBJECT() para construir el documento y JSON_EXTRACT() para consultar sus propiedades. |
| SQLite | ubicacion_gps TEXT. CHECK (ubicacion_gps IS NULL OR json_valid(ubicacion_gps)) valida el texto. json_extract() permite consultar sus propiedades. |

JSON permite incluir metadatos variables, como precisión o fuente, sin añadir una columna por cada dato opcional. Los datos centrales del evento siguen en columnas relacionales. Para guardar únicamente dos coordenadas fijas también serían válidas columnas numéricas; aquí se eligieron datos semiestructurados para comparar cómo los tres motores los manejan.

El caso se relaciona con IoT por la posible procedencia de una lectura de un dispositivo conectado. No se afirma que el proyecto ya sea una solución de big data. Para analizar muchas lecturas históricas haría falta ampliar el modelo con fecha de

captura, dispositivo y registros separados, además de estudiar volumen e indexación.

La validación actual comprueba la sintaxis JSON, pero no exige que existan latitud y longitud ni verifica sus rangos. Esa validación adicional queda como mejora. Incorporar JSON en una base relacional tampoco la convierte, por sí solo, en un SGBD NoSQL.

## Verificaciones, dificultades y ajustes

Las pruebas realizadas durante el desarrollo permitieron confirmar la creación de las tablas y el uso básico de JSON. No constituyen una validación exhaustiva de todas las operaciones.

| Comprobación | Resultado observado |
|---|---|
| Inventario del modelo | Se confirmaron 14 tablas del proyecto en los tres motores. |
| Estudiantes y eventos | Se aceptaron inserciones de prueba con un creador registrado. |
| JSON en los tres motores | Se registraron eventos con ubicación. En MySQL y SQLite se comprobaron las coordenadas mediante consultas de extracción; en PostgreSQL se confirmó el uso de JSONB. |
| Triggers de MySQL | Se verificaron los cuatro triggers. Las inserciones de autoseguimiento y automentoría fueron rechazadas; en conexion se observó el error 1644 con el mensaje definido. |
| Validaciones pendientes | Falta probar individualmente los triggers de UPDATE, cada cascada, RESTRICT, límites de longitud y entradas JSON inválidas. |

**Error 3823 de MySQL.** El CHECK que comparaba las dos cédulas entró en conflicto con las acciones referenciales de las FK. Se conservaron las cascadas y se implementaron validaciones BEFORE INSERT y BEFORE UPDATE con SIGNAL SQLSTATE '45000'. El CHECK de las fechas de mentoría se mantuvo porque opera sobre otros atributos.

## CONCLUSIONES

Al comenzar pensaba que, como las tablas ya funcionaban en PostgreSQL, adaptarlas a otro motor sería principalmente cambiar algunas palabras. Durante el proceso entendí que una regla puede necesitar una implementación distinta. El error de MySQL con CHECK y CASCADE fue el ejemplo más claro: tuve que entender los triggers y el mensaje generado con SIGNAL.

También comprendí mejor la diferencia entre generar un identificador y referenciar uno existente. En SQLite preferí utilizar enteros autoincrementales para simplificar el trabajo, pero eso me obligó a revisar las tablas relacionadas. Con JSON aprendí que declarar una columna prepara el espacio para los datos; después se necesita un INSERT para guardarlos y una consulta para revisarlos.

Este trabajo continuó lo que había construido desde el foro. Corregir mi propio código y mantener comentarios me ayudó a comprender los cambios. Como mejora, ampliaría las pruebas y revisaría que los scripts y diccionarios describan exactamente la misma versión.

# RECURSOS

Link del video en youtube https://www.youtube.com/watch?v=cJ7zQ8MVURI

Link del video en google drive
https://docs.google.com/videos/d/1lLmSkEDylYolkooWhPvh3gR62zPjKlWpP8otfRlL5pc/play?usp=sharing

DDL PostgreSQL
https://drive.google.com/file/d/1ZdtXF8Gvsz9xEB_xZ6z0aBDqM0DoyTOH/view?usp=sharing

DDL MySQL
https://drive.google.com/file/d/1M9fzy0WrDQKUljBXkrgatldk9H37tjrY/view?usp=sharing

DDL SQLite
https://drive.google.com/file/d/1rEWslVL1FgNLcJWQxC0B_fP06bcBJTTF/view?usp=sharing

Diccionario de Datos
https://docs.google.com/spreadsheets/d/1cMZ6xYztVji7wiOB80b5bLlXl3CmPsFI0aoiJGbWYRw/edit?usp=sharing